Testes do ATC

Testes do Advanced Trigonometry Calculator

Este documento resume a infraestrutura de testes do ATC 2.1.9.

Suite principal

Executar após compilar o ATC:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-regression.ps1 -AtcExe .\x64\Release\atc.exe
```

Resultado histórico em Release x64 e Release x86 (não aprova o candidato 2.1.9):

```
Summary: 377 passed, 0 failed
```

Cobertura isolada histórica:

```
Summary: 68 passed, 0 failed
```

Validação histórica do package SourceForge:

```
Summary: 44 passed, 0 failed
```

Cobertura automatizada

A suite cobre, entre outros:

- aritmética;
- syntax guards do parser para parenteses invalidos, argumentos vazios,

operadores consecutivos e operadores mal posicionados;

- trigonometria e funções hiperbolicas;
- logaritmos;
- numeros complexos;
- matrizes;
- estatística, DSP e probabilidade;
- simplificacao polinomial;
- solver e equation solver;
- function study;
- graph smoke tests;
- settings persistentes;
- autocomplete e histórico;
- TXT processing com mocks seguros;
- módulos interativos com caminhos smoke seguros.

Fluxo TXT

O fluxo direto `predefine txt` -> `solve txt` usa fixtures temporárias. A suite cobre aritmética, `solver`, `solve equation`, `simplify polynomial`, recuperacao após linha invalida e abertura mockada do ficheiro de respostas.

O watcher `auto solve txt` tem agora fixture runtime deterministica: usa um ficheiro TXT real com `SOLVE_NOW`, valida que a flag e consumida, confirma o ficheiro `_answers.txt` e regista a abertura do ficheiro por mock. `roots to polynomial` dentro de `solve txt` ainda deve ser tratado separadamente antes de entrar na regressão padrão.

Stress de memória

Para comandos sensíveis a memória:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-memory-stress.ps1 -Iterations 10
```

SRS Gate Runner

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-srs-gates.ps1 -AtcExe .\x64\Release\atc.exe -SkipStress
```

Este runner valida gates mensuráveis da SRS:

- Stage 1: parser e syntax guard com mais de 50 inputs;
- Stage 2: persistência de `higherprecision` e startup gate;
- Stage 3: polinómios de grau 20, solver, latência e Memory Factor;
- Stage 4: `auto solve txt`, `SOLVE_NOW`, `_answers.txt` e protecao de

diretórios não relacionados.

Validacao curta mais recente:

```
Summary: 4 passed, 0 failed
StressIterations: 2
```

Limites

Comandos que desligam/reiniciam o PC, abrem janelas externas ou esperam tempo real longo devem continuar fora da suite automática padrão até existirem mocks seguros.

Validacao do package SourceForge

O runner de validação do package e:

```
tests\run-atc-package-validation.ps1
```

Valida a estrutura preparada para SourceForge sem reconstruir o ZIP. A cobertura inclui ficheiros raiz, executaveis e launchers x64/x86, PDFs de documentação, ausencia de pastas `Source code` duplicadas, ausencia de ficheiros `_answers.txt` gerados, referências GitHub/source e consistencia SHA256 de todos os ficheiros empacotados.

Exemplo:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-package-validation.ps1
```

Ligacao entre documentação e testes

Comportamento documentado deve ter cobertura automática sempre que for deterministico e seguro com input redirecionado.

Regras ao atualizar documentação:

- exemplos estáveis devem ser adicionados a suite de regressão quando

praticavel;

- módulos interativos devem ter pelo menos smoke test seguro ou nota explicita

de validação manual;

- comandos que abrem janelas, ficheiros, browsers ou acoes de PC devem usar

mocks, checks source-level ou notas manuais;

- não atualizar a contagem documentada de testes sem executar a suite;
- atualizar `tests\ATC_USER_GUIDE_COVERAGE.md` quando uma area documentada

ganha ou perde cobertura.

O Cookbook e as Boas Práticas devem usar comandos já documentados. Se uma receita não puder ser testada com segurança, deve indicar isso nas notas ou limitacoes.

Validação de desempenho de scripts

O interpretador de scripts tem validação automática de desempenho com o exemplo `Scripts examples/Multiplication Table 1-100_script_generator.txt`. Este cenário exercita ciclos `for` aninhados, atribuições, reutilização de variáveis, `print("...", ...)` e avaliação repetida de expressões escalares.

Na versão 2.1.7, este fluxo reduz pressão de memória ao libertar arrays temporários do processamento TXT, evitar buffers matriciais em expressões escalares, executar linhas `print("...", ...)` seguras diretamente em memória e avaliar atribuições escalares simples, condições de ciclo e argumentos inteiros de `print` por um caminho leve do interpretador.

Executar o benchmark com:

```
powershell -ExecutionPolicy Bypass -File .\tests\run-atc-script-benchmark.ps1 -AtcExe .\x64\Release\atc.exe
```

O benchmark valida a saída do processo, tempo de execução, pico de memória RAM e conteúdo estável do output. Última validação Release x64:

```
Summary: 4 passed, 0 failed
ElapsedSeconds: 7.41
PeakWorkingSetMB: 119.56
```